Le coffre-fort de la salle NSI

Première NSI

Travail à distance — six fonctions Python à écrire et tester dans la page

Langages et programmation

Le lycée est fermé aujourd'hui. Le coffre-fort de la salle NSI, lui, attend : il est fermé par **six serrures**. Chaque serrure s'ouvre avec une fonction Python correcte, et chaque serrure ouverte révèle une lettre du code secret.

Objectifs

- Écrire une fonction qui **renvoie** un résultat avec `return`.
- Choisir entre plusieurs cas avec `if`, `and`, `or`.
- Répéter avec `for` (accumulateur) et `while` (condition d'arrêt).
- Lire un résultat de test et corriger son programme.

Prérequis

- Séquence SQ00 : variables, conditions, boucles, fonctions.
- Un téléphone ou un ordinateur avec une connexion Internet : rien à installer.

Sommaire

Étape	Contenu
Serrures 1 à 3	Renvoyer une valeur, choisir entre plusieurs cas
Serrures 4 et 5	Répéter avec for, puis avec while
Serrure 6 et bonus	Construire une chaîne de caractères

> **Repère**
>
> Écrivez votre code dans la zone grise, puis appuyez sur ▸ **Tester** : Python s'exécute dans la page et vérifie votre fonction sur plusieurs exemples. Une ligne ✕ indique l'appel qui échoue, la valeur attendue et celle obtenue. Le bouton **Exécuter** lance votre code sans les tests, pour essayer des `print`. Votre code reste enregistré sur cet appareil si vous fermez la page. Sur téléphone, le bouton ⇥ **Indenter** insère 4 espaces, et la touche Entrée reprend le décalage automatiquement après une ligne qui finit par `:`.

> **Attention**
>
> Travail personnel, **sans IA** : le but est que vous sachiez écrire ces fonctions lundi, seuls, dans Thonny. Si vous bloquez plus de 10 minutes sur une serrure, ouvrez l'indice dans les corrigés en bas de page, puis passez à la suivante.

Code du coffre

Serrure 1 — Le périmètre

Écrire la fonction `perimetre_carre(cote)` qui **renvoie** le périmètre d'un carré dont on donne la longueur du côté. Par exemple, `perimetre_carre(5)` vaut `20`.

```
def perimetre_carre(cote):
    # Renvoyer le périmètre du carré
    ...
```

```
perimetre_carre(5) == 20
perimetre_carre(0) == 0
perimetre_carre(2.5) == 10.0
```

Serrure 2 — Le tarif du musée

Un musée pratique trois tarifs : **5 €** pour les moins de 12 ans, **7 €** à partir de 65 ans, **10 €** pour tous les autres. Écrire la fonction `tarif(age)` qui renvoie le prix à payer.

Rappel : dès qu'un `return` est exécuté, la fonction s'arrête. Plusieurs `if` qui renvoient chacun leur résultat suffisent. Regardez bien les tests aux âges 12 et 65.

```
def tarif(age):
    ...
```

```
tarif(8) == 5
tarif(11) == 5
tarif(12) == 10
tarif(40) == 10
tarif(64) == 10
tarif(65) == 7
tarif(80) == 7
```

Serrure 3 — L'escape game

Un escape game accepte les joueurs de **16 ans ou plus**. Les plus jeunes ne peuvent entrer que s'ils sont **accompagnés** d'un adulte. Écrire la fonction `peut_entrer(age, accompagne)` qui renvoie `True` ou `False` ; le paramètre `accompagne` vaut lui-même `True` ou `False`.

Défi : une seule ligne `return` suffit, sans `if`.

```
def peut_entrer(age, accompagne):
    ...
```

```
peut_entrer(17, False) == True
peut_entrer(16, False) == True
peut_entrer(15, False) == False
peut_entrer(15, True) == True
peut_entrer(30, True) == True
```

Serrure 4 — La somme des entiers

Écrire la fonction `somme_jusqua(n)` qui renvoie la somme `1 + 2 + 3 + … + n`, calculée avec une boucle `for`. Par exemple, `somme_jusqua(4)` vaut `1 + 2 + 3 + 4 = 10`.

Pensez à l'**accumulateur** : une variable qui commence à `0` avant la boucle et augmente à chaque tour. Vérifiez quelles valeurs prend `i` dans `range(...)`.

```
def somme_jusqua(n):
    total = 0
    ...
```

```
somme_jusqua(1) == 1
somme_jusqua(4) == 10
somme_jusqua(10) == 55
```

```
somme_jusqua(0) == 0
somme_jusqua(100) == 5050
```

Serrure 5 — Les bactéries

Une population de bactéries **double toutes les heures**. Écrire la fonction `heures(depart, seuil)` qui renvoie le nombre d'heures nécessaires pour que la population, partie de `depart` bactéries, atteigne **au moins** `seuil`.

Exemple : en partant de 1 bactérie, on a 2, puis 4, puis 8, puis 16 : il faut **4 heures** pour atteindre au moins 10.

Le nombre de tours n'est pas connu à l'avance : c'est le travail d'une boucle `while`. Deux variables changent à chaque tour : la population et le nombre d'heures.

```
def heures(depart, seuil):
    population = depart
    nb_heures = 0
    ...
```

```
heures(1, 10) == 4
heures(1, 2) == 1
heures(1, 1) == 0
heures(3, 100) == 6
heures(1, 1000) == 10
```

Serrure 6 — Une ligne d'étoiles

Écrire la fonction `ligne(n)` qui **renvoie** (et n'affiche pas) une chaîne de `n` étoiles, construite avec une boucle, **sans** utiliser `"*" * n`. Par exemple, `ligne(3)` vaut `"***"`.

Même idée qu'un accumulateur, mais avec du texte : on part de la chaîne vide `""` et on ajoute une étoile à chaque tour avec `+`.

```
def ligne(n):
    resultat = ""
    ...
```

```
ligne(3) == "***"
ligne(1) == "*"
ligne(0) == ""
ligne(10) == "**********"
```

Bonus — Des étoiles espacées

Écrire `ligne_espacee(n)` qui renvoie `n` étoiles séparées par une espace, **sans espace à la fin** : `ligne_espacee(3)` vaut `"* * *"`. La première étoile, ou la dernière, est un cas particulier.

```
def ligne_espacee(n):
    ...
```

```
ligne_espacee(3) == "* * *"
ligne_espacee(1) == "*"
ligne_espacee(0) == ""
ligne_espacee(5) == "* * * * *"
```

Pour finir, affichez un escalier en **réutilisant** votre fonction `ligne` : complétez le programme, puis appuyez sur **Exécuter**.

```
def ligne(n):
    # Recopiez ici votre fonction de la serrure 6
    ...

for i in range(1, 6):
    print(ligne(i))
```

Avant lundi

Rien à envoyer. Notez sur papier ou dans un document le **code du coffre**, et pour la serrure qui vous a résisté le plus longtemps, l'erreur que vous faisiez et comment vous l'avez trouvée. Lundi, nous découvrirons ce que signifie le mot du coffre.

Corrigés

Première NSI

Travail à distance — six fonctions Python à écrire et tester dans la page

Langages et programmation

Ouvrez d'abord l'indice, réessayez, puis seulement la solution.

Serrure 1

Indice : un `print` affiche le résultat mais ne le renvoie pas ; le test attend un `return`.

```
def perimetre_carre(cote):
    return 4 * cote
```

Serrure 2

Indice : « moins de 12 ans » s'écrit `age < 12` ; « à partir de 65 ans » s'écrit `age >= 65`.

```
def tarif(age):
    if age < 12:
        return 5
    if age >= 65:
        return 7
    return 10
```

Serrure 3

Indice : on entre si l'âge suffit **ou** si l'on est accompagné.

```
def peut_entrer(age, accompagne):
    return age >= 16 or accompagne
```

Serrure 4

Indice : `range(n)` va de `0` à `n - 1` ; pour aller jusqu'à `n`, il faut `range(1, n + 1)`.

```
def somme_jusqua(n):
    total = 0
    for i in range(1, n + 1):
        total = total + i
    return total
```

Serrure 5

Indice : on continue tant que la population est **inférieure** au seuil. Si la page signale une boucle infinie, vérifiez que `population` change dans la boucle.

```
def heures(depart, seuil):
    population = depart
    nb_heures = 0
    while population < seuil:
```

```
        population = population * 2
        nb_heures = nb_heures + 1
    return nb_heures
```

Serrure 6

Indice : le `return` se place **après** la boucle, pas dedans.

```
def ligne(n):
    resultat = ""
    for i in range(n):
        resultat = resultat + "*"
    return resultat
```

Bonus

```
def ligne_espacee(n):
    resultat = ""
    for i in range(n):
        if i > 0:
            resultat = resultat + " "
        resultat = resultat + "*"
    return resultat
```